DATA STRUCTURES PRACTICALS

20 C++ Programs • Practical List 1–10 (A and B)

Prepared from the practical list provided. Each program is standalone: compile and run one program at a time.

Quick notes

• Save each program in a separate .cpp file and compile using a C++ compiler.
• For binary search, enter values in ascending order.
• Program 9A uses separate chaining as one common collision-handling technique.
• Program 10B uses Dijkstra’s algorithm and expects non-negative edge weights; enter 0 when there is no edge.

1A. 1-D Array: Searching, Sorting and Reversing (Menu Driven)

```
#include <iostream>
using namespace std;

int main() {
    int a[100], n, choice, key;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> a[i];

    do {
        cout << "\n1.Search  2.Sort ascending  3.Reverse  4.Display  0.Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            cout << "Enter element to search: ";
            cin >> key;
            int i;
            for (i = 0; i < n && a[i] != key; i++) {}
            if (i < n) cout << "Found at position " << i + 1 << "\n";
            else cout << "Element not found\n";
        } else if (choice == 2) {
            for (int i = 0; i < n - 1; i++)
                for (int j = 0; j < n - i - 1; j++)
                    if (a[j] > a[j + 1]) {
                        int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                    }
            cout << "Array sorted\n";
        } else if (choice == 3) {
            for (int i = 0; i < n / 2; i++) {
                int t = a[i]; a[i] = a[n - 1 - i]; a[n - 1 - i] = t;
            }
            cout << "Array reversed\n";
        } else if (choice == 4) {
            for (int i = 0; i < n; i++) cout << a[i] << " ";
            cout << "\n";
        }
    } while (choice != 0);
    return 0;
}
```

1B. Matrix Addition, Multiplication and Transpose (Menu Driven)

```
#include <iostream>
using namespace std;

int main() {
    int a[10][10], b[10][10], r, c, r2, c2, choice;
    cout << "Enter rows and columns of matrix A: ";
    cin >> r >> c;
    cout << "Enter matrix A:\n";
    for (int i=0;i<r;i++) for (int j=0;j<c;j++) cin >> a[i][j];

    do {
        cout << "\n1.Addition  2.Multiplication  3.Transpose of A  0.Exit\nChoice: ";
        cin >> choice;
        if (choice == 1) {
            cout << "Enter rows and columns of matrix B: ";
            cin >> r2 >> c2;
            if (r != r2 || c != c2) { cout << "Addition not possible\n"; continue; }
            cout << "Enter matrix B:\n";
            for (int i=0;i<r2;i++) for (int j=0;j<c2;j++) cin >> b[i][j];
            for (int i=0;i<r;i++) {
                for (int j=0;j<c;j++) cout << a[i][j]+b[i][j] << " ";
                cout << "\n";
            }
        } else if (choice == 2) {
            cout << "Enter rows and columns of matrix B: ";
            cin >> r2 >> c2;
            if (c != r2) { cout << "Multiplication not possible\n"; continue; }
            cout << "Enter matrix B:\n";
            for (int i=0;i<r2;i++) for (int j=0;j<c2;j++) cin >> b[i][j];
            int result[10][10] = {};
            for (int i=0;i<r;i++)
                for (int j=0;j<c2;j++)
                    for (int k=0;k<c;k++) result[i][j] += a[i][k]*b[k][j];
            for (int i=0;i<r;i++) {
                for (int j=0;j<c2;j++) cout << result[i][j] << " ";
                cout << "\n";
            }
        } else if (choice == 3) {
```

```
            for (int j=0;j<c;j++) {
                for (int i=0;i<r;i++) cout << a[i][j] << " ";
                cout << "\n";
            }
        }
    } while (choice != 0);
    return 0;
}
```

2A. Singly Linked List: Create and Display

```
#include <iostream>
using namespace std;

struct Node { int data; Node* next; };

int main() {
    Node *head = nullptr, *tail = nullptr;
    int n, value;
    cout << "How many nodes? ";
    cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: ";
        cin >> value;
        Node* node = new Node{value, nullptr};
        if (head == nullptr) head = tail = node;
        else { tail->next = node; tail = node; }
    }
    cout << "Linked list: ";
    for (Node* p=head; p!=nullptr; p=p->next) cout << p->data << " -> ";
    cout << "NULL\n";
    while (head != nullptr) { Node* temp=head; head=head->next; delete temp; }
    return 0;
}
```

2B. Singly Linked List: Search an Element

```
#include <iostream>
using namespace std;

struct Node { int data; Node* next; };

int main() {
    Node *head=nullptr, *tail=nullptr;
    int n, value, key, position=1;
    cout << "How many nodes? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        Node* node = new Node{value, nullptr};
        if (!head) head=tail=node;
        else { tail->next=node; tail=node; }
    }
    cout << "Enter value to search: "; cin >> key;
    Node* p=head;
    while (p && p->data != key) { p=p->next; position++; }
    if (p) cout << "Found at position " << position << "\n";
    else cout << "Element not found\n";
    while (head) { Node* temp=head; head=head->next; delete temp; }
    return 0;
}
```

3A. Doubly Linked List: Create and Display

```
#include <iostream>
using namespace std;

struct Node { int data; Node *prev, *next; };

int main() {
    Node *head=nullptr, *tail=nullptr;
    int n, value;
    cout << "How many nodes? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        Node* node = new Node{value, tail, nullptr};
        if (!head) head=node;
        else tail->next=node;
        tail=node;
    }
    cout << "Forward: ";
    for (Node* p=head;p;p=p->next) cout << p->data << " <-> ";
    cout << "NULL\nBackward: ";
```

```
    for (Node* p=tail;p;p=p->prev) cout << p->data << " <-> ";
    cout << "NULL\n";
    while (head) { Node* temp=head; head=head->next; delete temp; }
    return 0;
}
```

3B. Doubly Linked List: Search an Element

```
#include <iostream>
using namespace std;

struct Node { int data; Node *prev, *next; };

int main() {
    Node *head=nullptr, *tail=nullptr;
    int n, value, key, pos=1;
    cout << "How many nodes? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        Node* node = new Node{value, tail, nullptr};
        if (!head) head=node; else tail->next=node;
        tail=node;
    }
    cout << "Enter value to search: "; cin >> key;
    Node* p=head;
    while (p && p->data != key) { p=p->next; pos++; }
    if (p) cout << "Found at position " << pos << "\n";
    else cout << "Element not found\n";
    while (head) { Node* temp=head; head=head->next; delete temp; }
    return 0;
}
```

4A. Circular Linked List: Create and Display

```
#include <iostream>
using namespace std;

struct Node { int data; Node* next; };

int main() {
    Node *head=nullptr, *tail=nullptr;
    int n, value;
    cout << "How many nodes? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        Node* node = new Node{value, nullptr};
        if (!head) { head=tail=node; node->next=head; }
        else { tail->next=node; tail=node; tail->next=head; }
    }
    cout << "Circular list: ";
    if (head) {
        Node* p=head;
        do { cout << p->data << " -> "; p=p->next; } while (p != head);
        cout << "(back to head)\n";
    } else cout << "Empty\n";
    if (head) {
        tail->next=nullptr;
        while (head) { Node* temp=head; head=head->next; delete temp; }
    }
    return 0;
}
```

4B. Circular Linked List: Search an Element

```
#include <iostream>
using namespace std;

struct Node { int data; Node* next; };

int main() {
    Node *head=nullptr, *tail=nullptr;
    int n, value, key, pos=1;
    cout << "How many nodes? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        Node* node = new Node{value, nullptr};
        if (!head) { head=tail=node; node->next=head; }
        else { tail->next=node; tail=node; tail->next=head; }
    }
    cout << "Enter value to search: "; cin >> key;
    bool found=false;
    if (head) {
```

```
        Node* p=head;
        do {
            if (p->data == key) { found=true; break; }
            p=p->next; pos++;
        } while (p != head);
    }
    if (found) cout << "Found at position " << pos << "\n";
    else cout << "Element not found\n";
    if (head) {
        tail->next=nullptr;
        while (head) { Node* temp=head; head=head->next; delete temp; }
    }
    return 0;
}
```

5A. Stack Using Array: Push, Pop, Display and Exit

```
#include <iostream>
using namespace std;
const int MAX = 100;

int main() {
    int stack[MAX], top=-1, choice, value;
    do {
        cout << "\n1.Push  2.Pop  3.Display  0.Exit\nChoice: ";
        cin >> choice;
        if (choice == 1) {
            if (top == MAX-1) cout << "Stack overflow\n";
            else { cout << "Enter value: "; cin >> value; stack[++top]=value; }
        } else if (choice == 2) {
            if (top == -1) cout << "Stack underflow\n";
            else cout << "Popped: " << stack[top--] << "\n";
        } else if (choice == 3) {
            if (top == -1) cout << "Stack is empty\n";
            else for (int i=top;i>=0;i--) cout << stack[i] << " ";
            cout << "\n";
        }
    } while (choice != 0);
    return 0;
}
```

5B. Queue Using Array: Insert, Delete, Display and Exit

```
#include <iostream>
using namespace std;
const int MAX = 100;

int main() {
    int q[MAX], front=0, rear=-1, count=0, choice, value;
    do {
        cout << "\n1.Insert  2.Delete  3.Display  0.Exit\nChoice: ";
        cin >> choice;
        if (choice == 1) {
            if (count == MAX) cout << "Queue is full\n";
            else {
                cout << "Enter value: "; cin >> value;
                rear=(rear+1)%MAX; q[rear]=value; count++;
            }
        } else if (choice == 2) {
            if (count == 0) cout << "Queue is empty\n";
            else { cout << "Deleted: " << q[front] << "\n"; front=(front+1)%MAX; count--; }
        } else if (choice == 3) {
            if (count == 0) cout << "Queue is empty\n";
            else for (int i=0;i<count;i++) cout << q[(front+i)%MAX] << " ";
            cout << "\n";
        }
    } while (choice != 0);
    return 0;
}
```

6A. Selection Sort

```
#include <iostream>
using namespace std;

int main() {
    int a[100], n;
    cout << "Enter number of elements: "; cin >> n;
    cout << "Enter elements: ";
    for (int i=0;i<n;i++) cin >> a[i];
    for (int i=0;i<n-1;i++) {
        int minIndex=i;
```

```
        for (int j=i+1;j<n;j++)
            if (a[j] < a[minIndex]) minIndex=j;
        int temp=a[i]; a[i]=a[minIndex]; a[minIndex]=temp;
    }
    cout << "Sorted array: ";
    for (int i=0;i<n;i++) cout << a[i] << " ";
    cout << "\n";
    return 0;
}
```

6B. Insertion Sort

```
#include <iostream>
using namespace std;

int main() {
    int a[100], n;
    cout << "Enter number of elements: "; cin >> n;
    cout << "Enter elements: ";
    for (int i=0;i<n;i++) cin >> a[i];
    for (int i=1;i<n;i++) {
        int key=a[i], j=i-1;
        while (j>=0 && a[j]>key) { a[j+1]=a[j]; j--; }
        a[j+1]=key;
    }
    cout << "Sorted array: ";
    for (int i=0;i<n;i++) cout << a[i] << " ";
    cout << "\n";
    return 0;
}
```

7A. Sequential (Linear) Search

```
#include <iostream>
using namespace std;

int main() {
    int a[100], n, key, i;
    cout << "Enter number of elements: "; cin >> n;
    cout << "Enter elements: ";
    for (i=0;i<n;i++) cin >> a[i];
    cout << "Enter element to search: "; cin >> key;
    for (i=0;i<n && a[i]!=key;i++) {}
    if (i<n) cout << "Element found at position " << i+1 << "\n";
    else cout << "Element not found\n";
    return 0;
}
```

7B. Binary Search (Input Must Be Sorted)

```
#include <iostream>
using namespace std;

int main() {
    int a[100], n, key;
    cout << "Enter number of sorted elements: "; cin >> n;
    cout << "Enter elements in ascending order: ";
    for (int i=0;i<n;i++) cin >> a[i];
    cout << "Enter element to search: "; cin >> key;
    int low=0, high=n-1, found=-1;
    while (low<=high) {
        int mid=low+(high-low)/2;
        if (a[mid]==key) { found=mid; break; }
        else if (a[mid]<key) low=mid+1;
        else high=mid-1;
    }
    if (found!=-1) cout << "Found at position " << found+1 << "\n";
    else cout << "Element not found\n";
    return 0;
}
```

8A. Binary Tree: Inorder, Preorder and Postorder Traversal

```
#include <iostream>
using namespace std;

struct Node { int data; Node *left, *right; };

Node* createTree() {
    int value;
    cout << "Enter value (-1 for no node): ";
```

```
    cin >> value;
    if (value == -1) return nullptr;
    Node* node = new Node{value, nullptr, nullptr};
    cout << "Left child of " << value << ":\n";
    node->left=createTree();
    cout << "Right child of " << value << ":\n";
    node->right=createTree();
    return node;
}
void inorder(Node* p) {
    if (p) { inorder(p->left); cout << p->data << " "; inorder(p->right); }
}
void preorder(Node* p) {
    if (p) { cout << p->data << " "; preorder(p->left); preorder(p->right); }
}
void postorder(Node* p) {
    if (p) { postorder(p->left); postorder(p->right); cout << p->data << " "; }
}
void freeTree(Node* p) {
    if (p) { freeTree(p->left); freeTree(p->right); delete p; }
}
int main() {
    cout << "Create tree in preorder; enter -1 for an empty node.\n";
    Node* root=createTree();
    cout << "Inorder: "; inorder(root);
    cout << "\nPreorder: "; preorder(root);
    cout << "\nPostorder: "; postorder(root);
    cout << "\n";
    freeTree(root);
    return 0;
}
```

8B. Insert Elements into a Maximum Heap

```
#include <iostream>
using namespace std;

int main() {
    int heap[100], n=0, count, value;
    cout << "How many elements to insert? "; cin >> count;
    for (int k=0;k<count;k++) {
        cout << "Enter value: "; cin >> value;
        if (n == 100) { cout << "Heap is full\n"; break; }
        int i=n++;
        heap[i]=value;
        while (i>0) {
            int parent=(i-1)/2;
            if (heap[parent] >= heap[i]) break;
            int temp=heap[parent]; heap[parent]=heap[i]; heap[i]=temp;
            i=parent;
        }
    }
    cout << "Max heap array: ";
    for (int i=0;i<n;i++) cout << heap[i] << " ";
    cout << "\n";
    return 0;
}
```

9A. Hashing Collision Technique: Separate Chaining

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    const int SIZE=10;
    vector<int> table[SIZE];
    int n, value;
    cout << "How many values? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        int index=((value%SIZE)+SIZE)%SIZE;
        table[index].push_back(value);
    }
    cout << "Hash table (separate chaining):\n";
    for (int i=0;i<SIZE;i++) {
        cout << i << ": ";
        for (int value : table[i]) cout << value << " -> ";
        cout << "NULL\n";
    }
    return 0;
}
```

9B. Hashing Using Linear Probing

```
#include <iostream>
using namespace std;

int main() {
    const int SIZE=10;
    int table[SIZE];
    for (int i=0;i<SIZE;i++) table[i]=-1;
    int n, value;
    cout << "How many values (maximum 10)? "; cin >> n;
    for (int i=0;i<n;i++) {
        cout << "Enter value: "; cin >> value;
        int index=((value%SIZE)+SIZE)%SIZE, tries=0;
        while (table[index]!=-1 && tries<SIZE) {
            index=(index+1)%SIZE;
            tries++;
        }
        if (tries==SIZE) cout << "Hash table full; could not insert " << value << "\n";
        else table[index]=value;
    }
    cout << "Hash table:\n";
    for (int i=0;i<SIZE;i++)
        if (table[i]==-1) cout << i << ": empty\n";
        else cout << i << ": " << table[i] << "\n";
    return 0;
}
```

10A. Generate an Adjacency Matrix

```
#include <iostream>
using namespace std;

int main() {
    int n, edges, u, v;
    int adj[20][20]={0};
    cout << "Enter number of vertices (max 20): "; cin >> n;
    cout << "Enter number of edges: "; cin >> edges;
    cout << "Enter each edge as two vertex numbers (1 to " << n << "):\n";
    for (int i=0;i<edges;i++) {
        cin >> u >> v;
        if (u>=1 && u<=n && v>=1 && v<=n) {
            adj[u-1][v-1]=1;
            adj[v-1][u-1]=1; // undirected graph
        }
    }
    cout << "Adjacency matrix:\n  ";
    for (int i=1;i<=n;i++) cout << i << " ";
    cout << "\n";
    for (int i=0;i<n;i++) {
        cout << i+1 << " ";
        for (int j=0;j<n;j++) cout << adj[i][j] << " ";
        cout << "\n";
    }
    return 0;
}
```

10B. Shortest Path Using Dijkstra's Algorithm

```
#include <iostream>
#include <climits>
using namespace std;

int main() {
    const int MAX=20;
    int n, graph[MAX][MAX], source;
    bool visited[MAX]={false};
    int dist[MAX];
    cout << "Enter number of vertices (max 20): "; cin >> n;
    cout << "Enter weighted adjacency matrix (0 means no edge, diagonal 0):\n";
    for (int i=0;i<n;i++)
        for (int j=0;j<n;j++) cin >> graph[i][j];
    cout << "Enter source vertex (1 to " << n << "): "; cin >> source;
    source--;
    for (int i=0;i<n;i++) dist[i]=INT_MAX;
    dist[source]=0;

    for (int step=0;step<n;step++) {
        int u=-1, best=INT_MAX;
        for (int i=0;i<n;i++)
            if (!visited[i] && dist[i]<best) { best=dist[i]; u=i; }
        if (u==-1) break;
        visited[u]=true;
```

```
        for (int v=0;v<n;v++) {
            if (!visited[v] && graph[u][v]>0 && dist[u]!=INT_MAX &&
                dist[u]+graph[u][v]<dist[v])
                dist[v]=dist[u]+graph[u][v];
        }
    }
    cout << "Shortest distances from vertex " << source+1 << ":\n";
    for (int i=0;i<n;i++) {
        cout << "To " << i+1 << ": ";
        if (dist[i]==INT_MAX) cout << "Unreachable\n";
        else cout << dist[i] << "\n";
    }
    return 0;
}
```